(5) If $E \simeq E'$, then $s_{[E]} = s_{[E']}$.
(6) For any sequences E, E' and w, such that $E.w$ is an execution sequence, we have $E \simeq E'$ iff $E.w \simeq E'.w$.
(7) If p, q, and r are different processes, then if $next_{[E]}(p) \rightarrow_{E.p.r} next_{[E.p]}(r)$ and $next_{[E]}(p) \not\rightarrow_{E.p.q} next_{[E.p]}(q)$, then $next_{[E]}(p) \rightarrow_{E.p.q.r} next_{[E.p.q]}(r)$.

The first six properties should be obvious for any reasonable happens-before relation; the only non-obvious one would be the last one. Intuitively, Property (7) states that if, after some sequence E, the next step of p happens before the next step of r, then the step of p still happens before the step of r even when some step of another process, which is not dependent with p, is inserted between p and r. This property holds for all computation models in which the happens-before assignment is based on whether events access shared objects in potentially conflicting ways. As an example, p and q can be reading a shared variable that is written by r. Another example is when p sends a message that is received by r. If an intervening process q is independent with p, it cannot affect this message, therefore r still receives the same message. It also holds for several more refined happens-before assignments, which are based on actual values involved in accesses to shared objects. An example is provided by the program in Fig. 9 (Section 8).

Properties (4) and (5) of Definition 3.2 together imply, as a special case, that if e and e' are two consecutive events in E with $e \not\rightarrow_E e'$, then they can be swapped and the state after the two events remains the same.

3.3 Independence and Races

We now define independence between events of a computation. If $E.p$ and $E.w$ are both execution sequences, then $E \vdash p \diamond w$ denotes that $E.p.w$ is an execution sequence such that $next_{[E]}(p) \not\rightarrow E.p.we$ for any $e \in dom_{[E.p]}(w)$. In other words, $E \vdash p \diamond w$ states that the next event of p would not "happen before" any event in w in the execution sequence $E.p.w$. Intuitively, it means that p is independent with w after E. In the special case when w contains only one process q, then $E \vdash p \diamond q$ denotes that the next steps of p and q are independent after E. We use $E \nvdash p \diamond w$ to denote that $E.p.w$ is an execution sequence for which $E \vdash p \diamond w$ does not hold.

For an execution sequence E and an event $e \in dom(E)$, let:

- $pre(E, e)$ denote the prefix of E up to, but not including, the event e,
- $notdep(e, E)$ denote the sub-sequence of E consisting of the events that occur after e but do not "happen after" e (i.e., the events e' that occur after e such that $e \not\rightarrow E e'$).

A central concept in most DPOR algorithms is that of a race. Let e and e' be two events in $dom(E)$, where $e <_E e'$. We say that

- *e is in a race with e'*, denoted $e \lessdot_E e'$ if $\widehat{e} \neq \widehat{e'}$ (i.e., these are events from different processes) and $e \rightarrow_E e'$ and there is no event $e'' \in dom(E)$, different from e' and e, such that $e \rightarrow_E e'' \rightarrow_E e'$,
- *e is in a reversible race with e'*, denoted $e \precsim_E e'$, if $e \lessdot_E e'$ and in any equivalent execution sequence $E' \simeq E$ where e occurs immediately before e', then $\widehat{e'}$ was not blocked before the occurrence of e.

Intuitively, $e \lessdot_E e'$ denotes that e and e' are co-enabled, i.e., there is an equivalent execution sequence $E' \simeq E$ in which e and e' are adjacent. Moreover, $e \precsim_E e'$ denotes that e does not enable e', so that the order of e and e' can be reversed.

Whenever a DPOR algorithm detects a race, then it will check whether the events in the race can be executed in the reverse order. Since the events are related by the happens-before relation, this